

Wanderlust Project: Interview Questions and Answers

Answers are written so you can say them in your own words. Claim only what you actually built. For anything you have not built, say: "I haven't implemented it yet, but I would approach it like this."

Section 1: Project Questions

Q1. Tell me about your project.

Wanderlust is a full-stack listing application similar to Airbnb. I built it with Node.js, Express, MongoDB (Mongoose) and EJS for server-side rendering. Users can view all listings, open one listing, create a new listing, edit it and delete it, so it covers full CRUD. I used RESTful routes, method-override for PUT and DELETE, and ejs-mate for reusable layouts. Next, I plan to add validation, error handling, authentication and image upload.

Q2. Which technologies did you use and why?

Node.js and Express for the backend because they are lightweight and let me use JavaScript on both sides. MongoDB with Mongoose because listings are flexible, document-shaped data. EJS for server-side rendering because it is simple and I can generate HTML directly from database data.

Q3. Why did you choose MongoDB instead of SQL?

Listings are document-like data (title, description, image, price, location) with no complex joins. MongoDB stores this naturally as JSON-like documents, the schema can change easily as I add features, and it works well with JavaScript. For highly relational data, like payments or bookings with strict integrity, I would consider SQL.

Q4. What was the hardest problem you faced?

Pick a real one. A good example: "My `/listing/new` route kept failing because I had defined `/listing/:id` above it, so Express treated 'new' as an id. Once I understood that Express matches routes in order, I moved the new route above it and it worked."

Q5. What would you improve in this project?

First, error handling with a `wrapAsync` helper and a central error-handling middleware. Second, validation with Joi so invalid data never reaches the database. Third, better structure by moving routes into `express.Router()` files and controllers. After that, authentication, image upload with Multer and Cloudinary, and reviews as a second collection referencing the listing.

Section 2: Express and Node.js

Q6. What is Node.js?

Node.js is a JavaScript runtime built on Chrome's V8 engine that lets JavaScript run outside the browser. It uses an event-driven, non-blocking I/O model, which makes it good at handling many concurrent requests.

Q7. What is Express?

Express is a minimal web framework for Node.js. It provides routing, middleware support and helpers for handling requests and responses, so I don't write everything with the raw `http` module.

Q8. What is middleware? Which ones did you use?

Middleware is a function that runs between the request and the response. It receives `req`, `res` and `next`, and it can modify the request, end the response, or pass control on with `next()`. I used `express.urlencoded` to parse form data into `req.body`, `method-override` to support PUT and DELETE, and `express.static` to serve files from the public folder.

Q9. What does `express.urlencoded({ extended: true })` do?

It parses form data sent with `application/x-www-form-urlencoded` and puts it in `req.body`. With `extended: true`, it can parse nested objects, which is how `req.body.listing` works when my form fields are named `listing[title]`, `listing[price]` and so on.

Q10. Why did you use `method-override`?

HTML forms only support GET and POST. For update and delete I need PUT and DELETE, so I send a POST with `?_method=PUT` or `?_method=DELETE`, and `method-override` converts it to the right method before it reaches my route.

Q11. What is the difference between `req.params`, `req.query` and `req.body`?

`req.params` holds route parameters, like `:id` in `/listing/:id`. `req.query` holds the query string, like `?search=goa`. `req.body` holds data sent in the request body, such as form fields on POST.

Q12. Why is `/listing/new` defined before `/listing/:id`?

Express matches routes in order from top to bottom. If `/listing/:id` came first, a request to `/listing/new` would treat "new" as an id and try to find a listing with that id, which would cause an error.

Q13. What is REST? Is your API RESTful?

REST is an architectural style where URLs represent resources and HTTP methods represent actions: GET reads, POST creates, PUT or PATCH updates, DELETE removes. My app follows this pattern, but my route names are not fully consistent (`/listing` vs `/listings`), and I would clean that up to use one plural convention.

Q14. What is the difference between PUT and PATCH?

PUT replaces the whole resource. PATCH updates only the fields that are sent.

Q15. What is the difference between `res.send`, `res.render` and `res.redirect`?

`res.send` sends a raw response like text or JSON. `res.render` renders an EJS template with data and sends the HTML. `res.redirect` tells the browser to make a new request to a different URL. After create, update and delete I use `redirect` so a page refresh does not resubmit the form.

Q16. What is `next()` and what happens if you don't call it?

`next()` passes control to the next middleware or route. If a middleware neither calls `next()` nor sends a response, the request hangs.

Q17. How do you handle errors in Express?

Ideally with an error-handling middleware that has four parameters (`err`, `req`, `res`, `next`), placed after all routes, and a `wrapAsync` helper that catches rejected promises from async routes and forwards them with `next(err)`. In my current version I haven't added this yet, so a failed database call can crash or hang the request. That is my first planned improvement.

Q18. What is the difference between `app.use()` and `app.get()`?

`app.use()` runs for all HTTP methods (and matching path prefixes), and is mostly used for middleware. `app.get()` handles only GET requests to an exact route.

Section 3: MongoDB and Mongoose

Q19. What is MongoDB?

MongoDB is a NoSQL document database. Data is stored as flexible JSON-like documents (BSON) inside collections, instead of rows inside tables.

Q20. What is Mongoose?

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js. It adds schemas, validation and convenient query methods on top of the MongoDB driver.

Q21. What is the difference between a schema and a model?

A schema defines the structure of a document: fields, types, defaults and validation. A model is compiled from the schema and gives me methods to talk to the collection, like `find`, `findById` and `save`. In my project, `Listing` is the model.

Q22. What is the difference between `find`, `findOne` and `findById`?

`find` returns an array of all documents matching a condition; with `{}` it returns everything. `findOne` returns the first matching document. `findById` is a shortcut for finding one document by its `_id`.

Q23. What does `findByIdAndUpdate` return?

By default it returns the old document, from before the update. To get the updated one, I pass `{ new: true }`.

Q24. What happens if `findById` finds nothing?

It returns `null`, not an error. If I pass that to my EJS template, the page can crash when it reads properties of `null`. I should check for null and respond with a 404 or redirect with a message.

Q25. What is an index and when would you use one?

An index is a data structure that speeds up queries on a field. I would add one on fields I search or sort by often, like `location` or `price`. The trade-off is extra storage and slightly slower writes.

Q26. What is the difference between embedding and referencing?

Embedding stores related data inside the same document, good for data read together. Referencing stores an `ObjectId` pointing to another document, good for data that grows or is shared. For reviews on a listing I would use referencing, because the number of reviews can grow without limit.

Q27. How would you add search and pagination?

For search, read a query string like `?search=goa` and filter with Mongoose, for example with a regex or a text index. For pagination, read `?page=2` and use `.skip((page - 1) * limit).limit(limit)`.

Q28. What is `populate()`?

It replaces a stored `ObjectId` reference with the actual referenced document. For example, if a listing stores review ids, `populate("reviews")` loads the full review documents.

Q29. What is `_id` in MongoDB?

Every document gets a unique `_id` field automatically, an `ObjectId` by default. It works as the primary key.

Section 4: Async JavaScript

Q30. Why do you use `async/await` in your routes?

Database calls are asynchronous because they take time. `await` pauses inside the function until the promise resolves, without blocking the rest of the server, and the code reads like normal sequential code.

Q31. What is a Promise?

A Promise is an object representing a value that will be available later. It can be pending, fulfilled or rejected.

Q32. What happens if an awaited promise rejects and you have no try/catch?

In Express 4, the error is not caught automatically in async routes, so the request hangs or the process can emit an unhandled rejection. That is why I would use a `wrapAsync` helper or try/catch and forward errors to error middleware.

Section 5: EJS and Rendering

Q33. What is EJS?

EJS (Embedded JavaScript templates) is a template engine that lets me embed JavaScript inside HTML to generate dynamic pages on the server, for example looping through listings.

Q34. What is server-side rendering versus client-side rendering?

In server-side rendering, the server builds the full HTML using database data and sends it to the browser. It is good for SEO and simple pages. In client-side rendering, the browser receives mostly JavaScript and builds the page itself, like in React.

Q35. What is the difference between `<%= %>` and `<%- %>`?

`<%= %>` outputs escaped HTML, which protects against XSS. `<%- %>` outputs raw, unescaped HTML, so I use it only for trusted content such as layout includes.

Q36. What does ejs-mate do?

It adds layout support to EJS. I write the boilerplate, like the navbar and footer, once, and reuse it in every page instead of repeating it.

Q37. How do you pass data from the route to the template?

As the second argument of `res.render`. For example, `res.render("listings/index.ejs", { allListing })` makes `allListing` available inside the template.

Section 6: Security and Validation

Q38. What is XSS and how do you prevent it?

Cross-site scripting is when an attacker injects malicious script into a page that other users view. I prevent it by escaping output (using `<%= %>` in EJS) and validating or sanitizing user input.

Q39. Why do you need server-side validation if the form already has it?

Client-side validation can be bypassed easily, for example with Postman or the browser dev tools. The server must validate again, using Joi or Mongoose validators, because it is the only trusted place.

Q40. Why should you not hardcode the database URL?

It exposes credentials if the code is pushed to GitHub, and it makes switching between development and production hard. I would store it in a `.env` file, load it with `dotenv`, and add `.env` to `.gitignore`.

Q41. What is a NoSQL injection?

An attack where user input is used to alter a database query, for example sending an object like `{ "$ne": null }` instead of a string. Prevention includes validating input types and using libraries like `express-mongo-sanitize`.

Section 7: Follow-up Feature Questions

Q42. How would you add authentication?

I would create a `User` model, hash passwords with `bcrypt` before saving, and start a session or issue a JWT on login. Then I would write an `isLoggedIn` middleware to protect create, edit and delete routes. I would also add an `owner` field to each listing so only the owner can modify it.

Q43. What is the difference between authentication and authorization?

Authentication verifies who the user is (login). Authorization decides what that user is allowed to do (for example only the owner can edit a listing).

Q44. Session vs JWT?

With sessions, the server stores the session data and gives the client a cookie with a session id. With JWT, the token itself carries the user data, signed by the server, and the client sends it with each request. Sessions suit server-rendered apps like mine. JWTs suit stateless APIs and mobile clients.

Q45. How would you handle image uploads?

Use Multer to parse multipart form data, then upload the file to a cloud service like Cloudinary, and store the returned URL and filename in the listing document. Right now I store the image URL as a string.

Q46. How would you add reviews?

Create a `Review` schema with rating and comment, connect it to the listing using a reference, and use `populate()` to load reviews on the show page. When a listing is deleted, delete its reviews too, for example with a Mongoose middleware hook.

Q47. How would you deploy this project?

Use MongoDB Atlas for the database, move secrets to environment variables, and deploy the Node app on a platform like Render or Railway. Then update the connection string and check the app on the live URL.

Q48. How would you structure this project better?

Follow MVC: models in a `models` folder, routes in `routes` using `express.Router()`, controllers holding the handler logic, and views in `views`. This keeps `app.js` small and each file focused on one job.

Section 8: Code-Based Questions

Q49. Explain what happens when a user opens `/listing/123`.

Express matches the route `GET /listing/:id`. I read the id from `req.params`, call `Listing.findById(id)` to fetch the document from MongoDB, then call `res.render("listings/show.ejs", { listing })`. EJS builds the HTML with that data and the server sends it to the browser.

Q50. Explain what happens when a user submits the new listing form.

The browser sends a POST request to `/listings`. `express.urlencoded` parses the form body into `req.body`. I create a new `Listing` from `req.body.listing`, set the image object, save it with `await listing.save()`, and redirect to `/listing`, which shows the updated list.

Q51. How does the edit and update flow work?

`GET /listing/:id/edit` loads the listing and renders the edit form. The form submits a POST with `?_method=PUT`, method-override turns it into PUT, and the route calls `findByIdAndUpdate(id, { ...req.body.listing })` and redirects.

Q52. What does `{ ...req.body.listing }` do?

The spread operator copies all properties of the `listing` object from the form into a new object, which becomes the update data.

Q53. Any security concern in `findByIdAndUpdate(id, { ...req.body.listing })`?

Yes. It trusts whatever fields the client sends, so a user could try to update fields that should not be editable. I should validate the body and allow only the expected fields.

Q54. What is wrong with saving the image URL this way, and how would you improve it?

I set `listing.image` to an object after creating the document, and the form sends a plain string. It works, but it is fragile. I would define the image structure in the schema (with a default and a setter), validate it with Joi, and use real file upload later.

Quick Revision Checklist

- Explain your project in 30 seconds
- Middleware, `next()`, and the request-response cycle
- `params`, `query`, `body`
- REST, CRUD, PUT vs PATCH
- Schema vs model, `find` vs `findOne` vs `findById`
- `{ new: true }` in `findByIdAndUpdate`
- Embedding vs referencing, `populate()`
- `async/await` and error handling
- SSR vs CSR, `<%= %>` vs `<%- %>`
- Authentication vs authorization, session vs JWT
- Your improvement plan: error handling, validation, MVC structure, auth, image upload